

Web App Pentest Checklist

A methodology-first walkthrough for authorized web application assessments — recon through reporting. Pair with the Burp Suite field guide for tool-level detail.

01 Recon & Mapping

Passive recon

- ❑ **Confirm scope** in writing — hosts, IP ranges, subdomains, out-of-scope assets, rules of engagement, testing window. No scope confirmation, no testing.
- ❑ Enumerate subdomains and DNS records before touching the app directly.
- ❑ Check certificate transparency logs and WHOIS/ASN data for related infrastructure.
- ❑ Search for exposed source, secrets, or config in public repos and paste sites.
- ❑ Fingerprint the stack: server headers, cookies, error pages, JS bundle names, CMS/framework tells.

```
subfinder -d target.tld -silent | httpx -title -status-code -tech-detect  
# -> live hosts, page titles, status, detected tech in one pass
```

Active mapping

- ❑ Crawl authenticated and unauthenticated app states separately — a crawler with no session misses most of the app.
- ❑ Walk every role (anon, user, admin, tenant-scoped) and diff the resulting sitemaps.
- ❑ Pull JS bundles and grep for hidden routes, feature flags, and API paths.
- ❑ Brute-force common paths/backup files (`.git`, `.env`, `.bak`, admin panels) against scope, not blindly across the internet.
- ❑ Build a request inventory in your proxy (Burp/ZAP) grouped by feature, not just by URL.

02 Authentication

- ❑ Username/email enumeration via login, registration, password reset, and error-message timing differences.
- ❑ Password policy: minimum strength enforced, no client-side-only validation, no silent truncation.
- ❑ Brute-force / credential-stuffing protection: rate limiting, lockout, CAPTCHA — and that it applies per-account *and* per-source.
- ❑ Password reset token: single-use, short-lived, bound to the account, not leaked via Referer header or in a redirect URL.
- ❑ MFA: can it be bypassed by hitting the post-auth endpoint directly, replaying an old code, or downgrading to a non-MFA flow?
- ❑ OAuth/SSO: `state` parameter validated, `redirect_uri` allow-listed exactly (not just prefix-matched), token audience checked.
- ❑ JWT: algorithm confusion (`alg: none`), weak/guessable signing secret, missing expiry validation, key confusion (RS256 → HS256).
- ❑ "Remember me" and long-lived tokens don't outlive a password change or explicit logout.

03 Session Management

- ❑ Session identifiers are high-entropy, random, and never derived from predictable user data.

- ❑ Cookies: `Secure`, `HttpOnly`, and an appropriate `SameSite` value are all present — not just one of them.
- ❑ Session fixation: pre-auth session ID is invalidated and replaced on successful login.
- ❑ Logout actually invalidates the session server-side, not just the client-side cookie.
- ❑ Concurrent session handling matches the app's stated policy (single-session vs. multi-device).
- ❑ Idle and absolute session timeouts are enforced server-side.

04 Access Control & IDOR

Why this is the highest-yield section: broken access control is consistently the most common high-severity finding in real engagements, and it's rarely caught by automated scanners — every one of these needs to be tested by hand, per role, per object.

- ❑ **Horizontal:** swap object IDs (order, invoice, profile, ticket) between two accounts of the *same* role.
- ❑ **Vertical:** replay a low-privilege session against an admin/privileged endpoint or UI action.
- ❑ Test every HTTP verb per endpoint, not just the one the UI uses — a `GET` may be checked while `PATCH` / `DELETE` on the same route isn't.
- ❑ Test IDs that are sequential, UUID, and hashed/encoded the same way — encoding is not authorization.
- ❑ Multi-tenant apps: confirm tenant/org isolation on every list, search, and export endpoint, not just detail views.
- ❑ Force-browse directly to feature-flagged or "not yet visible" UI states.
- ❑ Check that access control is enforced server-side even when the client hides the button/menu item.

05 Input Validation & Injection

SQL / NoSQL injection CRIT

- Every parameter that reaches a query — including headers, cookies, and JSON body fields, not just the query string.
- Error-based, boolean-blind, and time-blind confirmation when error output is suppressed.
- NoSQL: operator injection (`$ne` , `$gt`) in JSON bodies where a string was expected.

```
sqlmap -u "https://target/app?id=1" --batch --level=3 --risk=1 --dbs
# confirm manually first - don't let the tool be your only evidence
```

Cross-site scripting

- Reflected: any request parameter echoed into the response without contextual encoding.
- Stored: content that persists and renders back to other users (comments, profile fields, filenames).
- DOM-based: trace `location` / `postMessage` / `document.referrer` into sinks like `innerHTML` , `eval` , `document.write` .
- Test inside every context separately: HTML body, attribute, JS string, URL, CSS — each needs different payload shaping.

Server-side injection

- **STTI**: template syntax (`{{*}}` , `$(*)`) in any field that might be rendered through a template engine.
- **Command injection**: anywhere the app shells out — file conversion, ping/traceroute utilities, image processing.
- **XXE**: XML parsers with external entities enabled; try file read and blind OOB exfil via DTD.
- **Deserialization**: any endpoint accepting a serialized object (Java, PHP, Python pickle, .NET ViewState).
- **Path traversal**: file-serving or upload/download endpoints — test encoded and double-encoded `../` .

06 SSRF & Request Forgery

- Any feature that fetches a URL server-side: webhooks, "import from URL," link previews, PDF/screenshot generators, avatar-from-URL.
- Reach cloud metadata endpoints (`169.254.169.254`) and internal-only hosts/ports from those features.
- Bypass naive filters: alternate IP encodings, redirects (301 to an internal host), DNS rebinding.
- **CSRF**: state-changing requests without a per-session anti-CSRF token, or one that isn't actually validated server-side.
- Confirm CORS: reflected `Access-Control-Allow-Origin` combined with `Allow-Credentials: true` is a finding, not a config note.

07 Business Logic

No scanner finds these. Think like the app's workflow, then break the order, the quantity, or the assumption it depends on.

- Price/quantity manipulation: negative values, decimal rounding, currency mismatch, coupon stacking.
- Race conditions on anything limited or single-use: coupon redemption, inventory, "claim once" actions — fire requests concurrently.
- Workflow/state bypass: skip a required step by calling a later endpoint directly (e.g. confirm payment without completing checkout).

- ❑ Rate limits and quotas actually apply per resource they claim to protect, not just at the API gateway edge.
- ❑ File/export size and type limits enforced server-side, not just in the upload widget.

08 File Upload

- ❑ Extension check bypass: double extensions, null-byte tricks, case variation, alternate stream (`::$DATA`).
- ❑ Content-type/magic-byte checks vs. actual parsed file type — upload a polyglot.
- ❑ Where uploads are served from: same origin as the app (stored XSS risk) vs. a properly isolated storage origin.
- ❑ Server-side processing of uploaded files (image resize, PDF parse, zip extract) for injected exploits (ImageTragick-style, zip-slip, XXE via SVG).
- ❑ Overwrite existing files via a crafted filename/path in the upload request.

09 API Testing (REST / GraphQL)

- ❑ Every endpoint from the OpenAPI/Swagger spec is tested, not only the ones the frontend calls.
- ❑ Mass assignment: send extra JSON fields (`role` , `isAdmin` , `balance`) and see if the backend binds them.
- ❑ API versioning: an old, less-restricted version of an endpoint still reachable.
- ❑ Rate limiting and pagination limits are enforced, and can't be bypassed by header/casing tricks.
- ❑ **GraphQL**: introspection enabled in production; batched queries bypassing rate limits; excessive nesting/query-depth DoS; field-level authorization gaps that object-level checks miss.

```
curl -s https://target/graphql -H 'Content-Type: application/json' \
  -d '{"query":"{__schema{types{name,fields{name}}}}"}' | jq .
# introspection left on -> full schema, including admin-only fields
```

10 Configuration & Transport

- ❑ TLS: no weak ciphers/protocols, valid cert chain, HSTS present with a real max-age.
- ❑ Security headers present and not just copy-pasted boilerplate: `CSP` , `X-Frame-Options` / `frame-ancestors` , `X-Content-Type-Options` .
- ❑ Verbose errors/stack traces disabled in the environment under test.
- ❑ Default credentials on any exposed admin/management interface.
- ❑ Directory listing, backup files, and source maps not exposed alongside production assets.
- ❑ Cloud storage buckets referenced by the app aren't publicly listable/writable.

11 Client-Side & Storage

- ❑ Sensitive data (tokens, PII) not persisted in `localStorage` where XSS would exfiltrate it directly.
- ❑ `postMessage` handlers validate `origin` before trusting message content.
- ❑ Third-party scripts and widgets scoped so a compromised one can't read the whole page's DOM/cookies.
- ❑ Clickjacking: sensitive actions can't be framed and clicked through an overlay.

12 Evidence & Reporting

- ❑ Every finding has: request/response pair, reproduction steps, impact tied to the specific app (not a generic CVSS blurb), and a concrete fix.
- ❑ Screenshots/PoCs redact other testers' and real users' data before they leave your machine.
- ❑ Chained findings (a low-severity info leak that enables a high-severity IDOR) are written up as a chain, not scattered as isolated low-severity items.
- ❑ Retest confirms the fix at the code/config level, not just that the original payload no longer fires.

13 Quick Reference — Common Payload Starters

CLASS	STARTER PAYLOAD	WATCH FOR
SQLi	' OR '1'='1	Error text, row-count/behavior change
SQLi (blind)	' AND SLEEP(5) --	Consistent response delay
XSS	"><svg onload=alert(1)>	Unescaped in response body
SSTI	{{7*7}} / \${7*7}	Rendered as "49"
XXE	<!ENTITY xxe SYSTEM "file:///etc/passwd">	File contents in response
Path traversal	..%2f..%2f..%2fetc%2fpasswd	File contents, 500 vs 404 diff
Command inj.	; id ; whoami	Command output, timing delay
CRLF inj.	%0d%0aSet-Cookie: x=1	Injected header in response

Every payload here is a starting probe, not a finding. Confirm impact, avoid destructive follow-through, and stay inside the agreed scope and rules of engagement at all times.

zephyrx.in — for authorized security testing and educational use only. Always operate within a signed scope of engagement and applicable law.